Factors

Data Structures

- vectors
- *`factors`*
- lists
- matrices
- arrays
- dataframes

Factors

Factors are a peculiar data type: although they look like simple `characters`, they are actually a type of `integer` vector with some additional properties.

Creating a factor

Factors can be created either by converting a character vector with the `as.factor()` command …

```
1 char_vect <- rep(c("hello","ciao", "hola"), each = 2)
2 char_vect
```

```
[1] "hello" "hello" "ciao"  "ciao"  "hola"  "hola"
```

```
1 my_fact <- as.factor(char_vect)
2 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

… or by explicitly creating a factor with the `factor()` command:

```
1 my_fact <- factor(x = char_vect)
2 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
```

Levels: ciao hello hola

Properties of factors

In practice, they assign a label to an integer value:

```
1 typeof(my_fact) # typeof determines the type or storage mo
```

```
[1] "integer"
```

```
1 as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

This does not hold for character vectors!

```
1 typeof(char_vect) # typeof determines the type or storage
```

```
[1] "character"
```

```
1 as.integer(char_vect)
```

```
[1] NA NA NA NA NA NA
```

```
1 char_vect
```

```
[1] "hello" "hello" "ciao"  "ciao"  "hola"  "hola"
```

Factors store `levels()` as metadata,

```
1 levels(my_fact)
```

```
[1] "ciao"  "hello" "hola"
```

regardless of which ones are actually present in the vector. For example, if I create a factor made only of the elements of my_fact different from ciao:

```
1 my_fact2 <- my_fact[my_fact!="ciao"]
```

The levels of my_fact2 will be the same as those of my_fact (ciao included), even though ciao is no longer present as an observation:

```
1 my_fact2
```

```
[1] hello hello hola  hola
Levels: ciao hello hola
```

```
1 levels(my_fact2)
```

```
[1] "ciao"  "hello" "hola"
```

However, we can drop the levels that are no longer needed with the `droplevels()` command:

```
1 # what would my_fact2 look like if I dropped the levels wi
2 droplevels(my_fact2)
```

```
[1] hello hello hola  hola
Levels: hello hola
```

```
1 # modify my_fact2 by removing the unused levels
2 my_fact2 <- droplevels(my_fact2)
3 my_fact2
```

```
[1] hello hello hola  hola
Levels: hello hola
```

We can also **rename** the levels of a factor…

```
1 my_fact # the factor I created
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

```
1 my_fact_lev <- my_fact # create another identical factor "
```

by accessing them with the `levels` command:

```
1 levels(my_fact_lev)
```

```
[1] "ciao"  "hello" "hola"
```

```
1 # rename the levels of the factor
2 levels(my_fact_lev) <- c("italian","english","spanish")
```

Now the two factors are:

```
1 my_fact_lev
```

```
[1] english english italian italian spanish spanish
```

```
Levels: italian english spanish
```

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

We can **both reorder** and **rename** the levels of a factor with the `factor()` function:

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

```
1 # by default levels are assigned in alphabetical order
2 as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

We can decide that the first level should be hello:

```
1 new_fact <- factor(x = my_fact, levels = c("hello","ciao",
2 new_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: hello ciao hola
```

```
1 as.integer(new_fact)
```

```
[1] 1 1 2 2 3 3
```

We can also rename the levels, again with the `factor()` function:

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

```
1 new_fact2 <- factor(x = my_fact, levels = c("hello","ciao"
2                     labels = c("english","italian","spanish
3 new_fact2
```

```
[1] english english italian italian spanish spanish
Levels: english italian spanish
```

Previous versions:

```
1 new_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: hello ciao hola
```

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

Remember: the ***`levels`*** argument inside the **`factor()`** function is used to **reorder** the levels of the factor, while the **`levels()`** function is used to access the level names and, if needed, **rename** them!

```
1 # the factor I created
2 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

```
1 levels(my_fact)
```

```
[1] "ciao"  "hello" "hola"
```

```
1 levels(my_fact) <- c("hello","ciao","hola")
```

What do you notice compared to the my_fact vector shown above?

```
1 my_fact
```

```
[1] ciao  ciao  hello hello hola  hola
Levels: hello ciao hola
```

```r
as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

Ordinal Factors

A subtype of factors are ***ordered factors***, which correspond to ordinal variables, i.e. variables whose levels are sorted in increasing order:

```
1 my_ord_fac <- as.ordered(my_fact)
2
3 my_ord_fac # alphabetical order
```

```
[1] ciao  ciao  hello hello hola  hola
Levels: hello < ciao < hola
```

```
1 attributes(my_ord_fac)
```

```
$levels
[1] "hello" "ciao"  "hola"

$class
[1] "ordered" "factor"
```

Let’s practice!

Open this link and keep it open:

https://etherpad.wikimedia.org/p/arca-corsoR